

SENTINEL: Kubernetes Pod Entropy Monitor

Local Development Blueprint

1. PROJECT VISION

The Problem

Containers can "drift" from their original state due to:

- Malicious actors installing unauthorized software
- Compromised processes spawning new binaries
- Configuration tampering
- Cryptominers or malware

Your Solution: SENTINEL

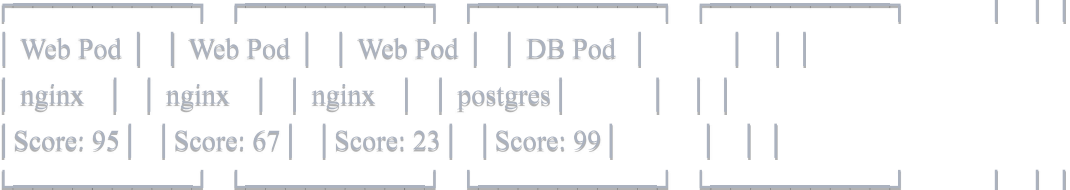
An **entropy-based scoring system** that:

1. Captures a "golden snapshot" of each container at startup
 2. Continuously monitors for deviations (drift)
 3. Calculates a health score (0-100, where 100 = pristine)
 4. Displays a real-time leaderboard
 5. Automatically purges "rotten" pods based on thresholds
-

2. LOCAL ARCHITECTURE

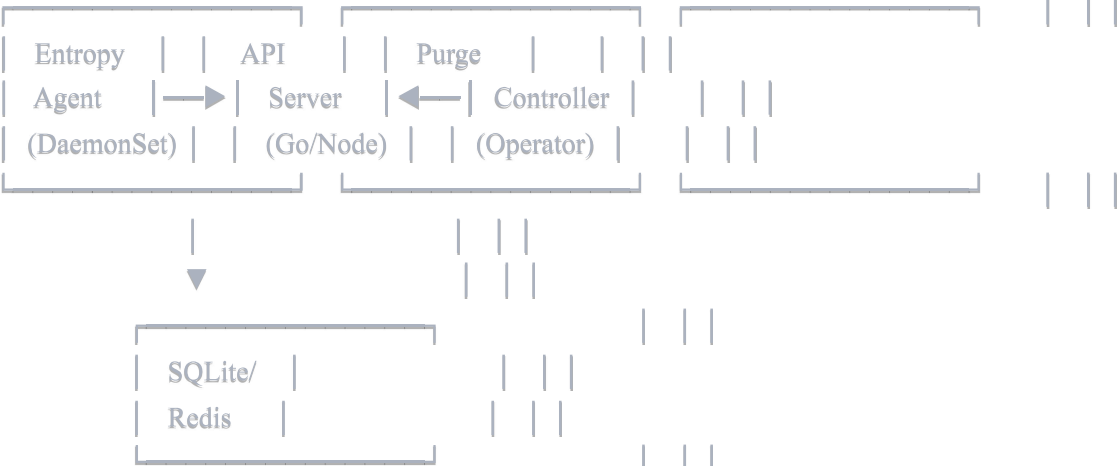


APP NAMESPACE (demo-app)



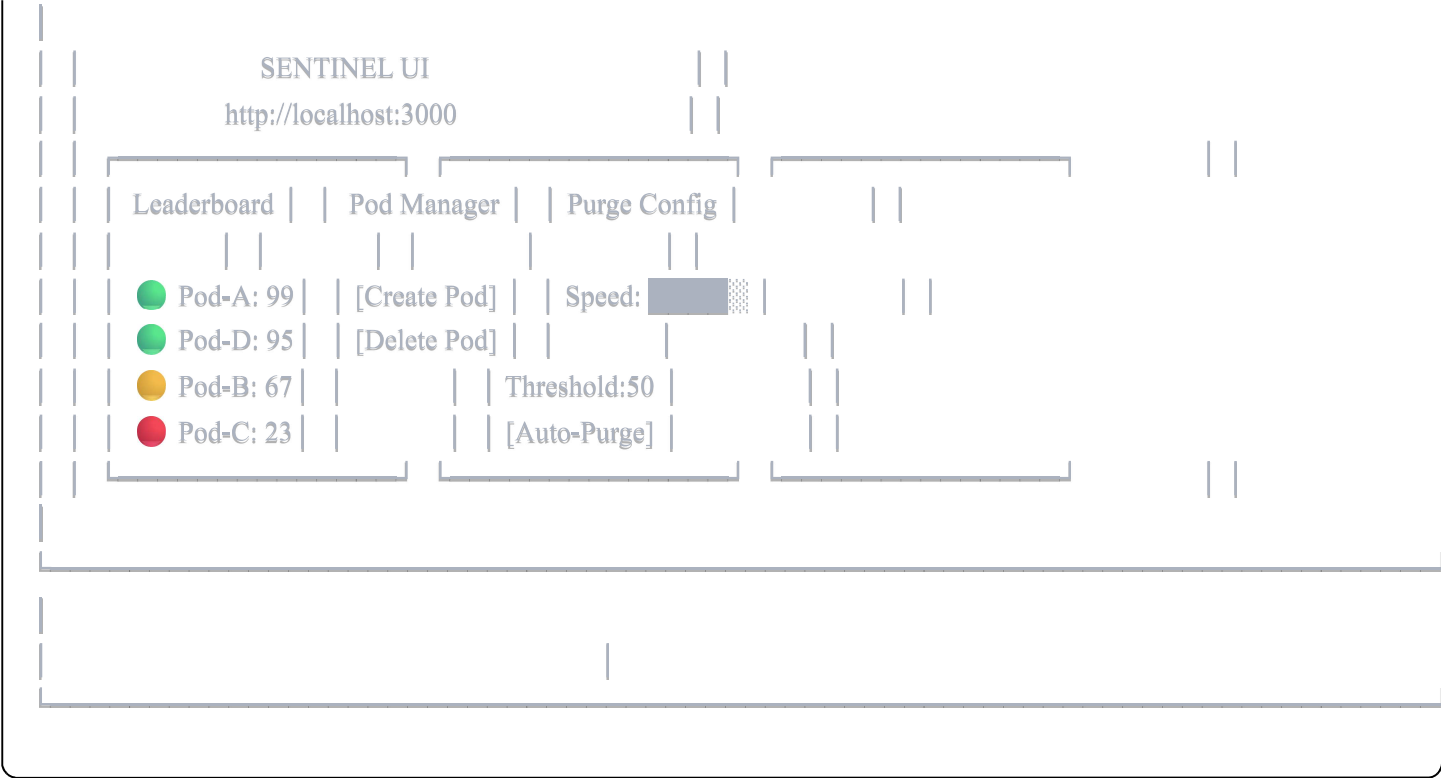
▲
monitors

SENTINEL NAMESPACE



port-forward / ingress





3. TECHNOLOGY STACK (Local-Optimized)

Infrastructure Layer

Component	Technology	Why
Local K8s	minikube or kind	Easy setup, works on Mac/Win/Linux
Container Runtime	Docker Desktop	Already on most dev machines
Local Registry	minikube registry	Push local images

Backend Services

Component	Technology	Why
Entropy Agent	Go	Low overhead, K8s native, can exec into containers
API Server	Go (Fiber) or Node.js (Fastify)	Fast, lightweight
Purge Controller	Go + client-go	Native K8s API
Database	SQLite (dev) or Redis	Zero config, embedded

Component	Technology	Why
Message Queue	Redis Pub/Sub or In-memory channels	Simple for local

Frontend

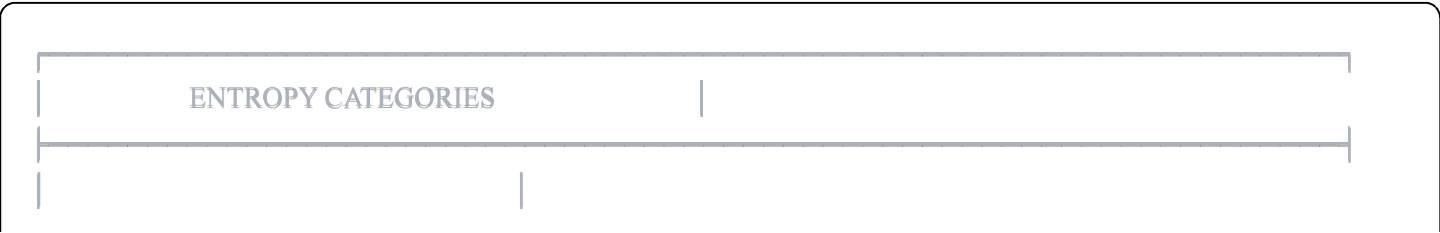
Component	Technology	Why
Framework	React + TypeScript	Standard, great tooling
Styling	Tailwind CSS	Rapid UI development
Components	shadcn/ui	Beautiful, accessible
State	React Query + Zustand	Server sync + local state
Charts	Recharts	Easy score visualization
Real-time	WebSocket or SSE	Live updates

Development Tools

Tool	Purpose
Tilt or Skaffold	Hot-reload K8s development
k9s	Terminal UI for K8s
Lens	Desktop K8s IDE
Docker Compose	Non-K8s local testing

4. ENTROPY SCORING SYSTEM

What We Monitor Per Container



FILESYSTEM (Weight: 35%) New files in /bin, /usr/bin, /sbin Modified executables (hash changes) New files in /tmp, /var/tmp Config file modifications	
PROCESSES (Weight: 30%) New running processes vs baseline Unexpected parent-child relationships Processes listening on new ports Crypto-mining signatures (high CPU processes)	
NETWORK (Weight: 20%) New listening ports Unexpected outbound connections DNS queries to suspicious domains	
PACKAGES (Weight: 10%) New installed packages (apt, apk, yum) Package manager invocations	
PERMISSIONS (Weight: 5%) New users/groups Sudo/privilege escalation File permission changes on sensitive files	

Score Calculation Formula

ENTROPY_SCORE = 100 - Σ(category_weight × drift_severity) Where drift_severity per category = min(100, Σ(event_severity_points)) Event Severity Points: New executable file: +15 points Modified system binary: +25 points New running process: +10 points New listening port: +20 points New installed package: +15 points New user created: +30 points

- Suspicious outbound conn: +20 points
- Config file modified: +5 points

Example Scoring

Pod "web-frontend-abc123" Baseline: Fresh nginx container

Events Detected:

- curl installed (apt-get): 15 pts → Packages
- New process: /tmp/miner: 10 pts → Processes
- /tmp/miner file created: 15 pts → Filesystem
- Port 4444 now listening: 20 pts → Network
- Modified /etc/passwd: 30 pts → Permissions

Category Scores:

- Filesystem: $100 - 15 = 85$ (weight 0.35) → 5.25 penalty
- Processes: $100 - 10 = 90$ (weight 0.30) → 3.00 penalty
- Network: $100 - 20 = 80$ (weight 0.20) → 4.00 penalty
- Packages: $100 - 15 = 85$ (weight 0.10) → 1.50 penalty
- Permissions: $100 - 30 = 70$ (weight 0.05) → 1.50 penalty

FINAL SCORE = $100 - (5.25 + 3.00 + 4.00 + 1.50 + 1.50) = 84.75 \rightarrow 85$

5. COMPONENT DETAILS

5.1 Entropy Agent (DaemonSet)

How it works:

1. Runs on each node as a DaemonSet
2. Uses `kubect exec` or CRI to inspect containers
3. On pod start: captures baseline snapshot
4. Every N seconds: compares current state to baseline
5. Reports drift events to API server

Baseline Snapshot Captures:

json

```

{
  "podId": "web-frontend-abc123",
  "containerId": "docker://...",
  "capturedAt": "2024-01-15T10:00:00Z",
  "filesystem": {
    "executableHashes": {
      "/bin/sh": "sha256:abc...",
      "/usr/bin/nginx": "sha256:def..."
    },
    "configHashes": {
      "/etc/nginx/nginx.conf": "sha256:ghi..."
    }
  },
  "processes": [
    { "pid": 1, "name": "nginx", "cmd": "nginx -g daemon off;" }
  ],
  "network": {
    "listeningPorts": [80, 443]
  },
  "packages": ["nginx", "libc"],
  "users": ["root", "nginx"]
}

```

5.2 API Server

Endpoints:

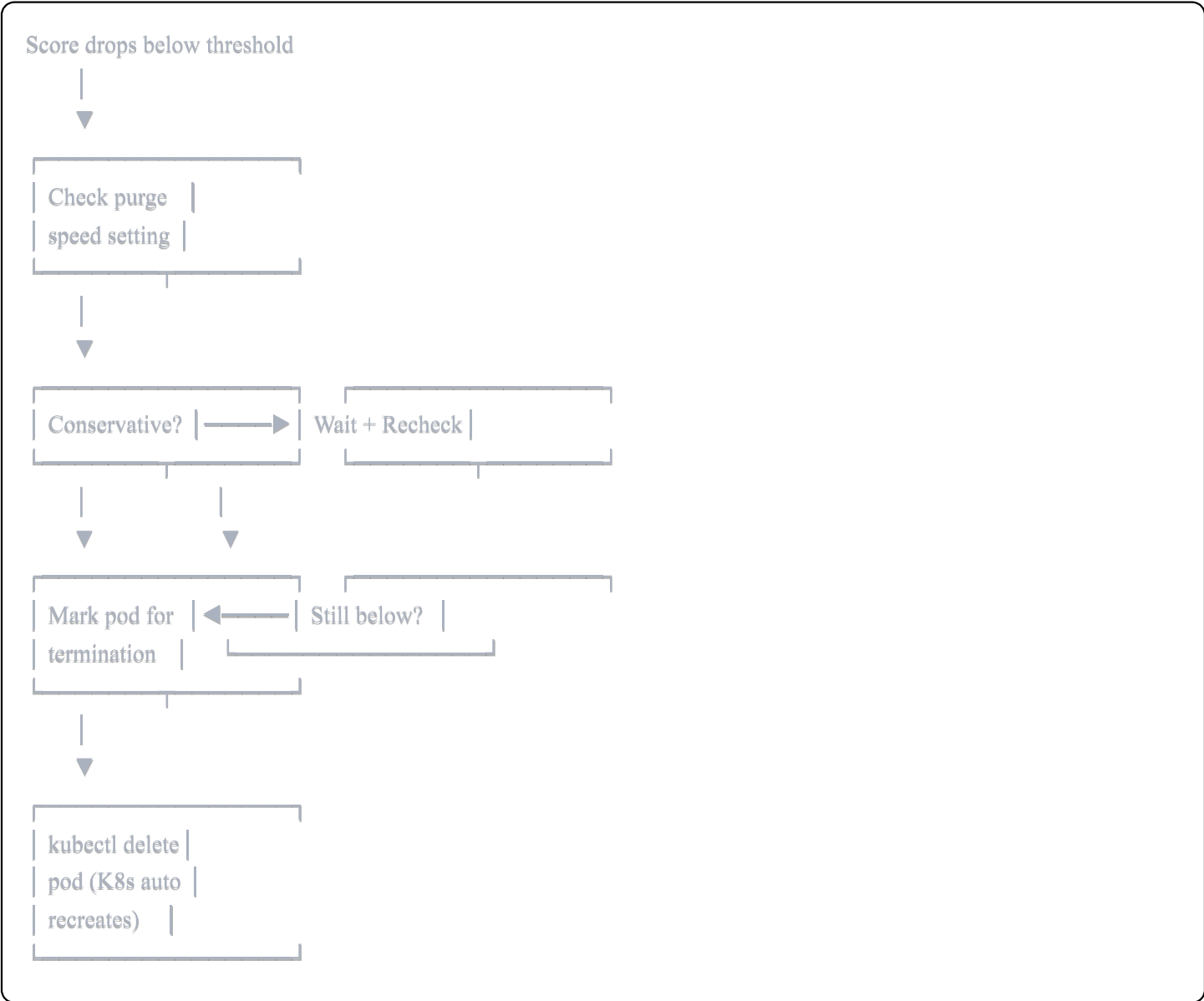
GET /api/pods	→ List all pods with scores
GET /api/pods/:id	→ Get pod details + drift events
GET /api/pods/:id/baseline	→ Get baseline snapshot
GET /api/leaderboard	→ Sorted pods by score
POST /api/pods	→ Create new pod (triggers K8s)
DELETE /api/pods/:id	→ Delete/purge pod
GET /api/config	→ Get purge configuration
PUT /api/config	→ Update purge config
WS /api/ws/scores	→ Real-time score stream

5.3 Purge Controller

Purge Speed Settings:

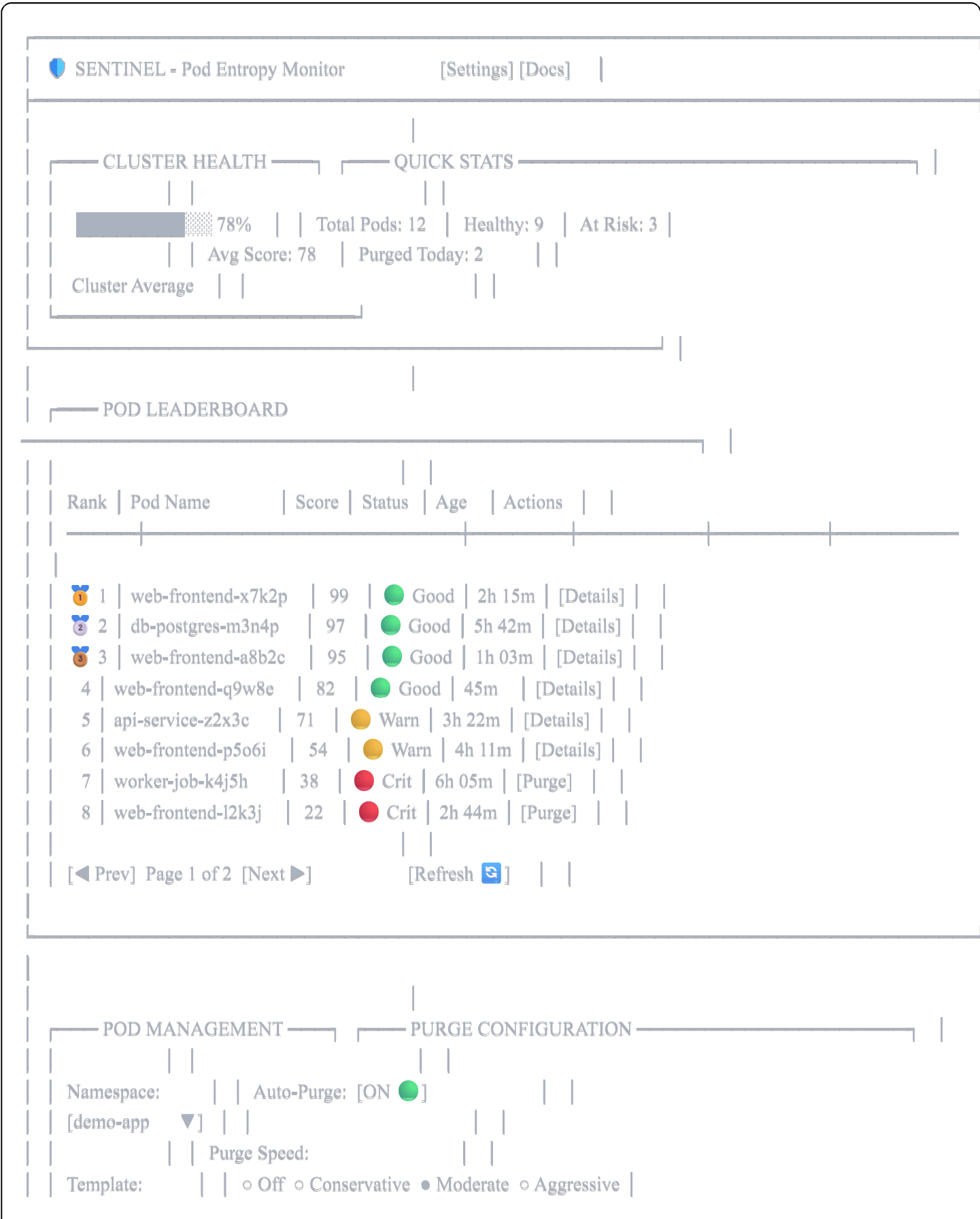
Speed	Threshold	Behavior
Off	-	No auto-purge, manual only
Conservative	<30	Alert, wait 5 min, re-check, then purge
Moderate	<40	Alert, wait 1 min, then purge
Aggressive	<50	Immediate purge + replace

Purge Flow:



6. UI WIREFRAMES

Main Dashboard Layout



[nginx-web ▼]

Score Threshold: [40] 40

Replicas: [3]

Grace Period: [60] seconds

Create Pod

Save Config

Pod Detail View

← Back to Leaderboard

POD: web-frontend-p5o6i

Score: 54/100

Status: Warning

Namespace: demo-app

Node: minikube

Age: 4h 11m

Last Check: 5s ago

Image: nginx:1.21

SCORE BREAKDOWN

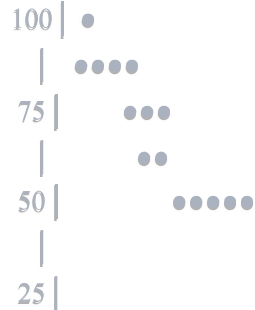
Filesystem	80%	↓ 20 from baseline
Processes	50%	↓ 50 from baseline
Network	70%	↓ 30 from baseline
Packages	40%	↓ 60 from baseline
Permissions	100%	No changes

DRIFT EVENTS (Last 1 hour)

🕒 10:45:22 🔴 HIGH New executable: /tmp/suspicious_script

🕒 10:42:15 🟡 MEDIUM Package installed: curl
🕒 10:40:03 🟡 MEDIUM New process: curl http://evil.com
🕒 10:38:44 🟢 LOW Config modified: /etc/nginx/nginx.conf
🕒 10:15:00 🟣 INFO Baseline captured

SCORE HISTORY



[🔄] Reset Baseline [⏸] Pause Monitoring [🗑] Purge Now

7. PROJECT STRUCTURE

```
sentinel/
├── README.md
├── docker-compose.yml      # For non-K8s local testing
├── Makefile                # Common commands
├── Tiltfile                # Hot-reload K8s dev
├──
├── k8s/                   # Kubernetes manifests
│   ├── demo-app/          # Sample app to monitor
│   │   └── namespace.yaml
```

- └─ nginx-deployment.yaml
- └─ postgres-deployment.yaml

- └─ sentinel/ # Sentinel components

- └─ namespace.yaml
- └─ agent-daemonset.yaml
- └─ api-deployment.yaml
- └─ controller-deployment.yaml
- └─ configmap.yaml

- └─ agent/ # Entropy Agent (Go)

- └─ Dockerfile
- └─ go.mod
- └─ main.go
- └─ pkg/
 - └─ baseline/ # Snapshot capture
 - └─ monitor/ # Drift detection
 - └─ scoring/ # Score calculation
 - └─ reporter/ # API communication
- └─ internal/
 - └─ container/ # Container inspection

- └─ api/ # API Server (Go or Node)

- └─ Dockerfile
- └─ main.go (or index.ts)
- └─ handlers/
- └─ models/
- └─ store/ # SQLite/Redis
- └─ websocket/

- └─ controller/ # Purge Controller (Go)

- └─ Dockerfile
- └─ main.go
- └─ pkg/
 - └─ purger/
 - └─ reconciler/

- └─ ui/ # React Frontend

- └─ Dockerfile
- └─ package.json
- └─ src/
 - └─ App.tsx
 - └─ components/
 - └─ Leaderboard/

```
| | | |— PodDetail/
| | | |— PodManager/
| | | |— PurgeConfig/
| | | |— Charts/
| | | |— hooks/
| | | |— api/
| | | |— types/
| | | |— tailwind.config.js
| | | |
| | | |— scripts/
| | | |— setup-minikube.sh
| | | |— deploy-all.sh
| | | |— simulate-attack.sh # Test script to create drift
```

8. DEVELOPMENT PHASES

Phase 1: Foundation (1-2 weeks)

- ☐ Set up minikube with demo app (nginx + postgres)
- ☐ Build basic agent that captures filesystem baseline
- ☐ Simple Go API server with SQLite
- ☐ CLI to view current scores

Deliverable: `kubectl get pods` shows pods, agent logs baselines

Phase 2: Core Monitoring (2-3 weeks)

- ☐ Full entropy agent (filesystem, processes, network, packages)
- ☐ Score calculation engine
- ☐ Drift event storage and retrieval
- ☐ WebSocket for real-time updates

Deliverable: Agent detects changes, API returns scores

Phase 3: UI Dashboard (2-3 weeks)

- ☐ React app with leaderboard
- ☐ Pod detail view with charts
- ☐ Pod creation form
- ☐ Purge configuration panel
- ☐ Real-time score updates

Deliverable: Functional dashboard at localhost:3000

Phase 4: Purge Controller (1-2 weeks)

- ☐ Kubernetes controller watching scores
- ☐ Auto-purge logic with configurable speeds
- ☐ Grace period handling
- ☐ Audit logging

Deliverable: Pods automatically purged when score drops

Phase 5: Polish (1 week)

- ☐ Tilt/Skaffold for dev experience
- ☐ Attack simulation scripts
- ☐ Documentation
- ☐ Demo video

Total: 7-11 weeks for full MVP

9. QUICK START COMMANDS

```
bash
```

1. Start minikube

```
minikube start --driver=docker --cpus=4 --memory=8g
```

2. Deploy demo application

```
kubectl apply -f k8s/demo-app/
```

3. Deploy Sentinel

```
kubectl apply -f k8s/sentinel/
```

4. Start UI development

```
cd ui && npm install && npm run dev
```

5. Port-forward API

```
kubectl port-forward -n sentinel svc/sentinel-api 8080:8080
```

6. Open dashboard

```
open http://localhost:3000
```

7. Simulate an attack (in another terminal)

```
./scripts/simulate-attack.sh
```

10. SIMULATING ATTACKS FOR TESTING

```
bash
```

```
#!/bin/bash
# scripts/simulate-attack.sh

# Get a random web pod
POD=$(kubectl get pods -n demo-app -l app=nginx -o jsonpath='{.items[0].metadata.name}')

echo "🎯 Target pod: $POD"

# Stage 1: Install package (score should drop ~15 points)
echo "📦 Installing curl..."
kubectl exec -n demo-app $POD -- apt-get update
kubectl exec -n demo-app $POD -- apt-get install -y curl

# Stage 2: Create suspicious file (score drops ~15 more)
echo "📁 Creating suspicious executable..."
kubectl exec -n demo-app $POD -- sh -c 'echo "#!/bin/sh\nnecho pwned" > /tmp/backdoor.sh'
kubectl exec -n demo-app $POD -- chmod +x /tmp/backdoor.sh

# Stage 3: Start new process (score drops ~10 more)
echo "⚙️ Starting suspicious process..."
kubectl exec -n demo-app $POD -- sh -c '/tmp/backdoor.sh &'

# Stage 4: Open new port (score drops ~20 more)
echo "🌐 Opening new network port..."
kubectl exec -n demo-app $POD -- apt-get install -y netcat-openbsd
kubectl exec -n demo-app $POD -- sh -c 'nc -l -p 4444 &'

echo "✅ Attack simulation complete!"
echo "🔍 Check the dashboard to see score changes"
```

11. RESOURCE REQUIREMENTS

Laptop Minimum Specs

Resource	Minimum	Recommended
RAM	8 GB	16 GB
CPU	4 cores	8 cores
Disk	20 GB free	50 GB free

Resource	Minimum	Recommended
OS	macOS/Linux/Win10+	macOS/Linux

Minikube Allocation

```
bash

minikube start \
  --cpus=4 \
  --memory=6g \
  --disk-size=30g \
  --driver=docker
```

12. NEXT STEPS

1. **Set up your environment** - Install Docker, minikube, kubectl, Go, Node.js
2. **Clone/create the project structure** - Use the layout above
3. **Start with the agent** - This is the core innovation
4. **Build incrementally** - Get each component working before moving on
5. **Test with attack simulation** - Verify your scoring works

Would you like me to start generating actual code for any of these components?